

Title

Project 1: E-Commerce Analytics Platform Using Apache Spark

Objective

The objective of this project is to develop an E-Commerce Analytics Platform using Apache Spark to process and analyze e-commerce transaction data efficiently. The project demonstrates how Spark DataFrames can be used for data ingestion, cleaning, transformation, ETL processing, business analytics, performance optimization, and report generation. It also illustrates the use of distributed data processing for analyzing large-scale datasets and generating meaningful business insights.

Project Information

This project implements a complete data analytics pipeline for an e-commerce platform using Apache Spark.

Three datasets are used in the project: **Customers**, **Products**, and **Orders**. These datasets are loaded into Spark DataFrames, cleaned by removing duplicate and null records, and then combined using ETL (Extract, Transform, Load) operations to create a unified Master Sales DataFrame.

The project performs various business analytics operations such as:

- Reading multiple CSV files using Spark.
- Data cleaning and validation.
- Removing duplicate and null records.
- ETL process using DataFrame joins.
- Creating a Master Sales DataFrame.
- Daily revenue analysis.
- Sales analysis by category.
- Sales analysis by city.
- Top-selling products analysis.
- Top customer analysis.
- Customer Lifetime Value (CLV) calculation.
- Average Order Value calculation.
- Performance optimization using Spark Cache and Broadcast Join.
- Exporting analytical reports into CSV files.
- Generating a final project summary.

The generated reports are automatically stored in the **output** directory, where each report is saved as a CSV file for further analysis or visualization.

ecommerce_analytics.py

```
from pyspark.sql import SparkSession
```

```
from pyspark.sql.functions import (
```

```
    col, when, count, sum, avg, desc, broadcast
```

```
)
```

```
# =====
```

```
# CREATE SPARK SESSION
```

```
# =====
```

```
spark = (
```

```
    SparkSession.builder
```

```
    .appName("ECommerce Analytics Platform")
```

```
    .master("local[*]")
```

```
    .config("spark.hadoop.fs.defaultFS", "file:///")
```

```
    .getOrCreate()
```

```
)
```

```
spark.sparkContext.setLogLevel("ERROR")
```

```
print("=" * 70)
```

```
print("E-COMMERCE ANALYTICS PLATFORM")
```

```
print("=" * 70)
```

```
# =====
```

```
# READ DATA
```

```
# =====
```

```
customers = (  
    spark.read  
    .option("header", True)  
    .option("inferSchema", True)  
    .csv("data/customers.csv")  
)
```

```
products = (  
    spark.read  
    .option("header", True)  
    .option("inferSchema", True)  
    .csv("data/products.csv")  
)
```

```
orders = (  
    spark.read  
    .option("header", True)  
    .option("inferSchema", True)  
    .csv("data/orders.csv")  
)
```

```
# =====
```

```
# DISPLAY DATA
```

```
# =====
```

```
print("\nCUSTOMERS")
customers.show()
```

```
print("\nPRODUCTS")
products.show()
```

```
print("\nORDERS")
orders.show()
```

```
# =====
# PRINT SCHEMA
# =====
```

```
print("\nCUSTOMER SCHEMA")
customers.printSchema()
```

```
print("\nPRODUCT SCHEMA")
products.printSchema()
```

```
print("\nORDER SCHEMA")
orders.printSchema()
```

```
# =====
# DATA CLEANING
# =====
```

```
print("=" * 70)
```

```
print("DATA CLEANING")
```

```
print("=" * 70)
```

```
print("\nOriginal Record Counts")
```

```
print("Customers :", customers.count())
```

```
print("Products  :", products.count())
```

```
print("Orders   :", orders.count())
```

```
customers = customers.dropDuplicates()
```

```
products = products.dropDuplicates()
```

```
orders = orders.dropDuplicates()
```

```
customers = customers.dropna()
```

```
products = products.dropna()
```

```
orders = orders.dropna()
```

```
print("\nRecord Counts After Cleaning")
```

```
print("Customers :", customers.count())
```

```
print("Products  :", products.count())
```

```
print("Orders   :", orders.count())
```

```
# =====
```

```
# NULL VALUE CHECK
```

```
# =====
```

```
print("=" * 70)
```

```
print("NULL VALUE CHECK")
```

```
print("=" * 70)
```

```
customers.select([
    count(when(col(c).isNull(), c)).alias(c)
    for c in customers.columns
]).show()
```

```
products.select([
    count(when(col(c).isNull(), c)).alias(c)
    for c in products.columns
]).show()
```

```
orders.select([
    count(when(col(c).isNull(), c)).alias(c)
    for c in orders.columns
]).show()
```

```
# =====
```

```
# ETL PROCESS
```

```
# =====
```

```
print("=" * 70)
```

```
print("ETL PROCESS - MASTER SALES DATA")
```

```
print("=" * 70)
```

```
sales = orders.join(  
    customers,  
    on="customer_id",  
    how="inner"  
)
```

```
sales = sales.join(  
    products,  
    on="product_id",  
    how="inner"  
)
```

```
print("\nMASTER SALES DATA")
```

```
sales.show(truncate=False)
```

```
print("\nMASTER DATA SCHEMA")
```

```
sales.printSchema()
```

```
# =====
```

```
# BUSINESS ANALYTICS
```

```
# =====
```

```
print("=" * 70)
```

```
print("BUSINESS ANALYTICS")
```

```
print("=" * 70)
```

```
# Daily Revenue
```

```
print("\n1. DAILY REVENUE")
```

```
daily_revenue = (  
    sales.groupBy("order_date")  
    .agg(sum("total_amount").alias("Daily_Revenue"))  
    .orderBy("order_date")  
)
```

```
daily_revenue.show()
```

```
# Sales by Category
```

```
print("\n2. SALES BY CATEGORY")
```

```
category_sales = (  
    sales.groupBy("category")  
    .agg(sum("total_amount").alias("Total_Sales"))  
    .orderBy(desc("Total_Sales"))  
)
```

```
category_sales.show()
```

```
# Sales by City
```



```
print("\n3. SALES BY CITY")
```

```
city_sales = (  
    sales.groupBy("city")  
    .agg(sum("total_amount").alias("Total_Sales"))  
    .orderBy(desc("Total_Sales"))  
)
```

```
city_sales.show()
```

```
# Top Selling Products
```

```
print("\n4. TOP SELLING PRODUCTS")
```

```
top_products = (  
    sales.groupBy("product_name")  
    .agg(  
        sum("quantity").alias("Total_Quantity"),  
        sum("total_amount").alias("Revenue")  
    )  
    .orderBy(desc("Revenue"))  
)
```

```
top_products.show()
```

```
# Top Customers
```

```
print("\n5. TOP CUSTOMERS")
```

```
top_customers = (  
    sales.groupBy("name")  
    .agg(sum("total_amount").alias("Total_Spent"))  
    .orderBy(desc("Total_Spent"))  
)
```

```
top_customers.show()
```

```
# Customer Lifetime Value
```

```
print("\n6. CUSTOMER LIFETIME VALUE")
```

```
clv = (  
    sales.groupBy("customer_id", "name")  
    .agg(  
        sum("total_amount").alias("Lifetime_Value"),  
        count("order_id").alias("Orders")  
    )  
    .orderBy(desc("Lifetime_Value"))  
)
```

```
clv.show()
```

```
# Average Order Value
```

```
print("\n7. AVERAGE ORDER VALUE")
```

```
sales.select(  
    avg("total_amount").alias("Average_Order_Value")  
).show()
```

```
# =====
```

```
# PERFORMANCE OPTIMIZATION
```

```
# =====
```

```
print("=" * 70)
```

```
print("PERFORMANCE OPTIMIZATION")
```

```
print("=" * 70)
```

```
sales.cache()
```

```
print("\nSales DataFrame Cached Successfully")
```

```
sales.count()
```

```
broadcast_products = broadcast(products)
```

```
optimized_sales = (  
    orders.join(customers, "customer_id")  
    .join(broadcast_products, "product_id")  
)
```

```
print("\nBroadcast Join Completed")
```

```
print("\nExecution Plan")
```

```
optimized_sales.explain()
```

```
# =====
```

```
# SAVE REPORTS
```

```
# =====
```

```
print("=" * 70)
```

```
print("SAVING REPORTS")
```

```
print("=" * 70)
```

```
daily_revenue.write.mode("overwrite").option(  
    "header", True  
).csv("output/daily_revenue")
```

```
category_sales.write.mode("overwrite").option(  
    "header", True  
).csv("output/category_sales")
```

```
city_sales.write.mode("overwrite").option(  
    "header", True  
).csv("output/city_sales")
```

```
top_products.write.mode("overwrite").option(  
    "header", True  
).csv("output/top_products")
```

```

        "header", True
    ).csv("output/top_products")

top_customers.write.mode("overwrite").option(
    "header", True
).csv("output/top_customers")

clv.write.mode("overwrite").option(
    "header", True
).csv("output/customer_lifetime_value")

print("\nAll Reports Saved Successfully")

# =====
# PROJECT SUMMARY
# =====

print("=" * 70)
print("PROJECT SUMMARY")
print("=" * 70)

print(f"Customers      : {customers.count()}")
print(f"Products       : {products.count()}")
print(f"Orders         : {orders.count()}")
print(f"Master Sales Rows : {sales.count()}")

print("\nProject Completed Successfully!")

```

spark.stop()

Screenshots:

```
sky@MS97377: ~/spark_exercises/eCommerce_project
/home/sky/.hushlogin file.
sky@MS97377:~/spark_exercises$ cd ~/spark_exercises
sky@MS97377:~/spark_exercises$ pwd
/home/sky/spark_exercises
sky@MS97377:~/spark_exercises$ mkdir eCommerce_project
sky@MS97377:~/spark_exercises$ cd eCommerce_project
sky@MS97377:~/spark_exercises/eCommerce_project$ pwd
/home/sky/spark_exercises/eCommerce_project
sky@MS97377:~/spark_exercises/eCommerce_project$ mkdir data output
sky@MS97377:~/spark_exercises/eCommerce_project$ ls
data output
sky@MS97377:~/spark_exercises/eCommerce_project$ nano data/customers.csv
sky@MS97377:~/spark_exercises/eCommerce_project$ nano data/products.csv
sky@MS97377:~/spark_exercises/eCommerce_project$ nano data/orders.csv
sky@MS97377:~/spark_exercises/eCommerce_project$ ls data
customers.csv orders.csv products.csv
sky@MS97377:~/spark_exercises/eCommerce_project$ cat data/customers.csv
customer_id,name,city
1,Alice,New York
2,Bob,Chicago
3,Charlie,Los Angeles
4,David,Houston
5,Eva,Miami
sky@MS97377:~/spark_exercises/eCommerce_project$ cat data/products.csv
product_id,product_name,category,price
101,Laptop,Electronics,800
102,Phone,Electronics,600
103,Shoes,Fashion,120
104,TShirt,Fashion,40
105,Watch,Accessories,250
sky@MS97377:~/spark_exercises/eCommerce_project$ cat data/orders.csv
order_id,customer_id,product_id,quantity,total_amount,order_date
1001,1,101,1,800,2026-01-01
1002,2,102,2,1200,2026-01-01
1003,1,103,3,360,2026-01-02
1004,3,101,1,800,2026-01-02
1005,4,104,5,200,2026-01-03
1006,5,105,2,500,2026-01-03
1007,2,103,1,120,2026-01-04
1008,3,102,1,600,2026-01-04
1009,4,101,2,1600,2026-01-05
1010,5,104,4,160,2026-01-05
sky@MS97377:~/spark_exercises/eCommerce_project$ nano eCommerce_analytics.py
sky@MS97377:~/spark_exercises/eCommerce_project$ python eCommerce_analytics.py
sky@MS97377:~/spark_exercises/eCommerce_project$ cd ~/spark_exercises
sky@MS97377:~/spark_exercises$ source venv/bin/activate
(venv) sky@MS97377:~/spark_exercises$ cd eCommerce_project
(venv) sky@MS97377:~/spark_exercises/eCommerce_project$ python -c "import pyspark; print(pyspark.__version__)"
4.1.1
(venv) sky@MS97377:~/spark_exercises/eCommerce_project$ python eCommerce_analytics.py
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
26/07/30 10:56:26 WARN Utils: Your hostname, MS97377, resolves to a loopback address: 127.0.1.1; using 172.29.63.103 instead (on interface eth0)
26/07/30 10:56:26 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/07/30 10:56:32 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
=====
E-COMMERCE ANALYTICS PLATFORM
=====
CUSTOMERS
=====
|customer_id|name|city|
|-----|
|1|Alice|New York|
|2|Bob|Chicago|
|3|Charlie|Los Angeles|
|4|David|Houston|
|5|Eva|Miami|
=====
PRODUCTS
=====
|product_id|product_name|category|price|
|-----|
|101|Laptop|Electronics|800|
|102|Phone|Electronics|600|
|103|Shoes|Fashion|120|
|104|TShirt|Fashion|40|
|105|Watch|Accessories|250|
=====
ORDERS
```

```
sky@MS9737: ~/spark_exercises/ecommerce_project
ORDERS
+-----+-----+-----+-----+-----+-----+
|order_id|customer_id|product_id|quantity|total_amount|order_date|
+-----+-----+-----+-----+-----+-----+
|1001|1|101|1|800|2026-01-01|
|1002|2|102|2|1200|2026-01-01|
|1003|1|103|3|360|2026-01-02|
|1004|3|101|1|800|2026-01-02|
|1005|4|104|5|200|2026-01-03|
|1006|5|105|2|500|2026-01-03|
|1007|2|103|1|120|2026-01-04|
|1008|3|102|1|600|2026-01-04|
|1009|4|101|2|1600|2026-01-05|
|1010|5|104|4|160|2026-01-05|
+-----+-----+-----+-----+-----+-----+

CUSTOMER SCHEMA
root
|-- customer_id: integer (nullable = true)
|-- name: string (nullable = true)
|-- city: string (nullable = true)

PRODUCT SCHEMA
root
|-- product_id: integer (nullable = true)
|-- product_name: string (nullable = true)
|-- category: string (nullable = true)
|-- price: integer (nullable = true)

ORDER SCHEMA
root
|-- order_id: integer (nullable = true)
|-- customer_id: integer (nullable = true)
|-- product_id: integer (nullable = true)
|-- quantity: integer (nullable = true)
|-- total_amount: integer (nullable = true)
|-- order_date: date (nullable = true)

=====
DATA CLEANING
=====

sky@MS9737: ~/spark_exercises/ecommerce_project
Original Record Counts
Customers : 5
Products : 5
Orders : 10

Record Counts After Cleaning
Customers : 5
Products : 5
Orders : 10

=====
NULL VALUE CHECK
=====
+-----+-----+-----+
|customer_id|name|city|
+-----+-----+-----+
|0|0|0|
+-----+-----+-----+

+-----+-----+-----+-----+-----+
|product_id|product_name|category|price|
+-----+-----+-----+-----+-----+
|0|0|0|0|
+-----+-----+-----+-----+

+-----+-----+-----+-----+-----+-----+
|order_id|customer_id|product_id|quantity|total_amount|order_date|
+-----+-----+-----+-----+-----+-----+
|0|0|0|0|0|0|
+-----+-----+-----+-----+-----+-----+

=====
ETL PROCESS - MASTER SALES DATA
=====

MASTER SALES DATA
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|product_id|customer_id|order_id|quantity|total_amount|order_date|name|city|product_name|category|price|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|101|1|1001|1|800|2026-01-01|Alice|New York|Laptop|Electronics|800|
|101|4|1009|2|1600|2026-01-05|David|Houston|Laptop|Electronics|800|
|103|1|1003|3|360|2026-01-02|Alice|New York|Shoes|Fashion|120|
|104|5|1010|4|160|2026-01-05|Eva|Miami|TShirt|Fashion|40|
|105|5|1006|2|500|2026-01-03|Eva|Miami|Watch|Accessories|250|
|102|3|1008|1|600|2026-01-04|Charlie|Los Angeles|Phone|Electronics|600|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
```

```

$ cd ~/MS97377; ./spark_exercises/ecommerce_project

1005      5      1006      2      500      2026-01-03|Eva      Watch      Accessories|250
1002      3      1008      1      600      2026-01-04|Charlie|Los Angeles|Phone      Electronics|600
1003      2      1007      1      120      2026-01-04|Bob      Chicago      Shoes      Fashion|120
1001      3      1004      1      800      2026-01-02|Charlie|Los Angeles|Laptop      Electronics|800
1002      2      1002      2      1200      2026-01-01|Bob      Chicago      Phone      Electronics|600
1004      4      1005      5      200      2026-01-03|David      Houston      TShirt      Fashion|40

=====
MASTER DATA SCHEMA
root
|-- product_id: integer (nullable = true)
|-- customer_id: integer (nullable = true)
|-- order_id: integer (nullable = true)
|-- quantity: integer (nullable = true)
|-- total_amount: integer (nullable = true)
|-- order_date: date (nullable = true)
|-- name: string (nullable = true)
|-- city: string (nullable = true)
|-- product_name: string (nullable = true)
|-- category: string (nullable = true)
|-- price: integer (nullable = true)

=====
BUSINESS ANALYTICS
=====

1. DAILY REVENUE
=====+
|order_date|Daily_Revenue|
=====+
|2026-01-01|      2000|
|2026-01-02|      1160|
|2026-01-03|      700|
|2026-01-04|      720|
|2026-01-05|      1760|
=====+

2. SALES BY CATEGORY
=====+
|category|Total_Sales|
=====+
|Electronics|      5000|
=====+

```


sky@MS97377: ~/spark_exercises/ecommerce_project

2. SALES BY CATEGORY

category	Total_Sales
Electronics	5000
Fashion	840
Accessories	500

3. SALES BY CITY

city	Total_Sales
Houston	1800
Los Angeles	1400
Chicago	1320
New York	1160
Miami	660

4. TOP SELLING PRODUCTS

product_name	Total_Quantity	Revenue
Laptop	4	3200
Phone	3	1800
Watch	2	500
Shoes	4	480
TShirt	9	360

5. TOP CUSTOMERS

name	Total_Spent
David	1800
Charlie	1400
Bob	1320
Alice	1160
Eva	660

```

sky@MS9737: ~/spark_exercises/ecommerce_project

6. CUSTOMER LIFETIME VALUE (CLV)
-----+-----+-----+-----+
|customer_id|name|Lifetime_Value|Orders|
-----+-----+-----+-----+
|4|David|1800|2|
|3|Charlie|1400|2|
|2|Bob|1320|2|
|1|Alice|1160|2|
|5|Eva|660|2|
-----+-----+-----+-----+

7. AVERAGE ORDER VALUE
-----+-----+-----+-----+
|Average_Order_Value|
-----+-----+-----+-----+
|634.0|
-----+-----+-----+-----+

=====
PERFORMANCE OPTIMIZATION
=====

Sales DataFrame has been cached.

Broadcast Join Completed Successfully.

Execution Plan
== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- InMemoryTableScan [product_id#60, customer_id#59, order_id#58, quantity#61, total_amount#62, order_date#63, name#18, city#19, product_name#38, category#39, price#40]
   +- InMemoryRelation [product_id#60, customer_id#59, order_id#58, quantity#61, total_amount#62, order_date#63, name#18, city#19, product_name#38, category#39, price#40], StorageLevel(disk, memory, deserialized, 1 replicas)
      +- AdaptiveSparkPlan isFinalPlan=true
         +- == Final Plan ==
            ResultQueryStage 5
            +- *(6) Project [product_id#60, customer_id#59, order_id#58, quantity#61, total_amount#62, order_date#63, name#18, city#19, product_name#38, category#39, price#40]
               +- *(6) BroadcastHashJoin [product_id#60], [product_id#37], Inner, BuildRight, false
                  :- *(6) Project [customer_id#59, order_id#58, product_id#60, quantity#61, total_amount#62, order_date#63, name#18, city#19]
                     : +- *(6) BroadcastHashJoin [customer_id#59], [customer_id#17], Inner, BuildRight, false
                        : :- *(6) HashAggregate(keys=[customer_id#59, order_date#63, product_id#60, total_amount#62, quantity#61, order_id#58], functions=[])

sky@MS9737: ~/spark_exercises/ecommerce_project
+- == Final Plan ==
   ResultQueryStage 5
   +- *(6) Project [product_id#60, customer_id#59, order_id#58, quantity#61, total_amount#62, order_date#63, name#18, city#19, product_name#38, category#39, price#40]
      +- *(6) BroadcastHashJoin [product_id#60], [product_id#37], Inner, BuildRight, false
         :- *(6) Project [customer_id#59, order_id#58, product_id#60, quantity#61, total_amount#62, order_date#63, name#18, city#19]
            : +- *(6) BroadcastHashJoin [customer_id#59], [customer_id#17], Inner, BuildRight, false
               : :- *(6) HashAggregate(keys=[customer_id#59, order_date#63, product_id#60, total_amount#62, quantity#61, order_id#58], functions=[])
                  : +- ShuffleQueryStage 0
                     : +- Exchange hashpartitioning(customer_id#59, order_date#63, product_id#60, total_amount#62, quantity#61, order_id#58, 200), ENSURE_REQUIREMENTS, [plan_id=4461]
                        : +- *(1) HashAggregate(keys=[customer_id#59, order_date#63, product_id#60, total_amount#62, quantity#61, order_id#58], functions=[])
                           : +- *(1) Filter ((atleastnonnulls(6, order_id#58, customer_id#59, product_id#60, quantity#61, total_amount#62, order_date#63) AND isNotNull(customer_id#59)) AND isNotNull(product_id#60))
                              : +- FileScan csv [order_id#58,customer_id#59,product_id#60,quantity#61,total_amount#62,order_date#63] Batched: false, DataFilters: [atleastnonnulls(6, order_id#58, customer_id#59, product_id#60, quantity#61, total_amount#62, or..., Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/home/sky/spark_exercises/ecommerce_project/data/orders.csv], PartitionFilters: [], PushedFilters: [IsNotNull(customer_id), IsNotNull(product_id)], ReadSchema: struct<order_id:int,customer_id:int,product_id:int,quantity:int,total_amount:int,order_date:date>
                                 +- BroadcastQueryStage 3
                                    +- BroadcastExchange HashedRelationBroadcastMode(List(cast(input[0, int, true] as bigint)),false), [plan_id=4588]
                                       +- *(4) HashAggregate(keys=[customer_id#17, name#18, city#19], functions=[])
                                          +- AQEShuffleRead coalesced
                                             +- ShuffleQueryStage 1
                                                +- Exchange hashpartitioning(customer_id#17, name#18, city#19, 200), ENSURE_REQUIREMENTS, [plan_id=4486]
                                                   +- *(2) HashAggregate(keys=[customer_id#17, name#18, city#19], functions=[])
                                                      +- *(2) Filter (atleastnonnulls(3, customer_id#17, name#18, city#19) AND isNotNull(customer_id#17))
                                                         +- FileScan csv [customer_id#17,name#18,city#19] Batched: false, DataFilters: [atleastnonnulls(3, customer_id#17, name#18, city#19), isNotNull(customer_id#17)], Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/home/sky/spark_exercises/ecommerce_project/data/customers.csv], PartitionFilters: [], PushedFilters: [IsNotNull(customer_id)], ReadSchema: struct<customer_id:int,name:string,city:string>
                                                            +- BroadcastQueryStage 4
                                                               +- BroadcastExchange HashedRelationBroadcastMode(List(cast(input[0, int, true] as bigint)),false), [plan_id=4626]
                                                                  +- *(5) HashAggregate(keys=[product_id#37, product_name#38, category#39, price#40], functions=[])
                                                                     +- AQEShuffleRead coalesced
                                                                        +- ShuffleQueryStage 2
                                                                           +- Exchange hashpartitioning(product_id#37, product_name#38, category#39, price#40, 200), ENSURE_REQUIREMENTS, [plan_id=4514]
                                                                              +- *(3) HashAggregate(keys=[product_id#37, product_name#38, category#39, price#40], functions=[])
                                                                                 +- *(3) Filter (atleastnonnulls(4, product_id#37, product_name#38, category#39, price#40) AND isNotNull(product_id#37))
                                                                                    +- FileScan csv [product_id#37,product_name#38,category#39,price#40] Batched: false, DataFilters: [atleastnonnulls(4, product_id#37, product_name#38, category#39, price#40), isNotNull(product_id...), Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/home/sky/spark_exercises/ecommerce_project/data/products.csv], PartitionFilters: [], PushedFilters: [IsNotNull(product_id)], ReadSchema: struct<product_id:int,product_name:string,category:string,price:int>
                                                                                       +- == Initial Plan ==
                                                                                          Project [product_id#60, customer_id#59, order_id#58, quantity#61, total_amount#62, order_date#63, name#18, city#19, product_name#38, category#39, price#40]

```

```

sky@MS97377: ~/spark_exercises/eCommerce_project
product_id#37, product_name#38, category#39, price#40), isNotNull(product_id..., Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/home/sky/spark_exercises/eCommerce_project/data/products.csv], PartitionFilters: [], PushedFilters: [IsNotNull(product_id)], ReadSchema: struct<product_id:int,product_name:string,category:string,price:int>

+- == Initial Plan ==
Project [product_id#60, customer_id#59, order_id#58, quantity#61, total_amount#62, order_date#63, name#18, city#19, product_name#38, category#39, price#40]

+- BroadcastHashJoin [product_id#60], [product_id#37], Inner, BuildRight, false
:- Project [customer_id#59, order_id#58, product_id#60, quantity#61, total_amount#62, order_date#63, name#18, city#19]
:- +- BroadcastHashJoin [customer_id#59], [customer_id#17], Inner, BuildRight, false
:- :- HashAggregate(keys=[customer_id#59, order_date#63, product_id#60, total_amount#62, quantity#61, order_id#58], functions=[])
:- :- +- Exchange hashpartitioning(customer_id#59, order_date#63, product_id#60, total_amount#62, quantity#61, order_id#58, 200), ENSURE_REQUIREMENTS, [plan_id=4415]
:- :- +- HashAggregate(keys=[customer_id#59, order_date#63, product_id#60, total_amount#62, quantity#61, order_id#58], functions=[])
:- :- +- Filter ((atleastnonnulls(6, order_id#58, customer_id#59, product_id#60, quantity#61, total_amount#62, order_date#63) AND isNotNull(customer_id#59)) AND isNotNull(product_id#60))
:- :- +- FileScan csv [order_id#58,customer_id#59,product_id#60,quantity#61,total_amount#62,order_date#63] Batched: false, DataFilters: [atleastnonnulls(6, order_id#58, customer_id#59, product_id#60, quantity#61, total_amount#62, or..., Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/home/sky/spark_exercises/eCommerce_project/data/orders.csv], PartitionFilters: [], PushedFilters: [IsNotNull(customer_id), IsNotNull(product_id)], ReadSchema: struct<order_id:int,customer_id:int,product_id:int,quantity:int,total_amount:int,order_date:date>
:- :- +- BroadcastExchange HashedRelationBroadcastMode(List(cast(input[0, int, true] as bigint)),false), [plan_id=4420]
:- :- +- HashAggregate(keys=[customer_id#17, name#18, city#19], functions=[])
:- :- +- Exchange hashpartitioning(customer_id#17, name#18, city#19, 200), ENSURE_REQUIREMENTS, [plan_id=4417]
:- :- +- HashAggregate(keys=[customer_id#17, name#18, city#19], functions=[])
:- :- +- Filter (atleastnonnulls(3, customer_id#17, name#18, city#19) AND isNotNull(customer_id#17))
:- :- +- FileScan csv [customer_id#17,name#18,city#19] Batched: false, DataFilters: [atleastnonnulls(3, customer_id#17, name#18, city#19), isNotNull(customer_id#17)], Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/home/sky/spark_exercises/eCommerce_project/data/customers.csv], PartitionFilters: [], PushedFilters: [IsNotNull(customer_id)], ReadSchema: struct<customer_id:int,name:string,city:string>
+- BroadcastExchange HashedRelationBroadcastMode(List(cast(input[0, int, true] as bigint)),false), [plan_id=4426]
+- HashAggregate(keys=[product_id#37, product_name#38, category#39, price#40], functions=[])
+- Exchange hashpartitioning(product_id#37, product_name#38, category#39, price#40, 200), ENSURE_REQUIREMENTS, [plan_id=4423]
+- HashAggregate(keys=[product_id#37, product_name#38, category#39, price#40], functions=[])
+- Filter (atleastnonnulls(4, product_id#37, product_name#38, category#39, price#40) AND isNotNull(product_id#37))
+- FileScan csv [product_id#37,product_name#38,category#39,price#40] Batched: false, DataFilters: [atleastnonnulls(4, product_id#37, product_name#38, category#39, price#40), isNotNull(product_id..., Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/home/sky/spark_exercises/eCommerce_project/data/products.csv], PartitionFilters: [], PushedFilters: [IsNotNull(product_id)], ReadSchema: struct<product_id:int,product_name:string,category:string,price:int>

=====
SAVING REPORTS
=====

All reports saved successfully!
=====
PROJECT SUMMARY
=====

sky@MS97377: ~/spark_exercises/eCommerce_project

All reports saved successfully!
=====
PROJECT SUMMARY
=====
Customers      : 5
Products       : 5
Orders         : 10
Master Sales Rows : 10

Project Completed Successfully!
(venv) sky@MS97377:~/spark_exercises/eCommerce_project$ ls output
category_sales customer_lifetime_value top_customers
city_sales      daily_revenue          top_products
(venv) sky@MS97377:~/spark_exercises/eCommerce_project$ ls output/daily_revenue
SUCCESS part-00000-c5d65d39-d22b-4b4c-85f8-d82451661169-c000.csv
(venv) sky@MS97377:~/spark_exercises/eCommerce_project$ ls output/city_sales
SUCCESS part-00000-36ba8ee2-40fc-4728-af9a-54854b4cc88e-c000.csv
(venv) sky@MS97377:~/spark_exercises/eCommerce_project$ ls output/daily_revenue
SUCCESS part-00000-c5d65d39-d22b-4b4c-85f8-d82451661169-c000.csv
(venv) sky@MS97377:~/spark_exercises/eCommerce_project$ cat output/daily_revenue/part-*.csv
order_date,daily_revenue
2026-01-01,2000
2026-01-02,1160
2026-01-03,700
2026-01-04,720
2026-01-05,1760
(venv) sky@MS97377:~/spark_exercises/eCommerce_project$ ls output/city_sales
SUCCESS part-00000-36ba8ee2-40fc-4728-af9a-54854b4cc88e-c000.csv
(venv) sky@MS97377:~/spark_exercises/eCommerce_project$ cat output/city_sales/part-*.csv
city,Totalsales
Houston,1800
Los Angeles,1400
Chicago,1320
New York,1160
Miami,660
(venv) sky@MS97377:~/spark_exercises/eCommerce_project$ ls output/category_sales
SUCCESS part-00000-20de7302-2012-4636-9895-23083334750d-c000.csv
(venv) sky@MS97377:~/spark_exercises/eCommerce_project$ cat output/category_sales/part-*.csv
category,Totalsales
Electronics,5000
Fashion,840
Accessories,500

```

```

sky@MS97377: ~/spark_exercises/ecommerce_project
(venv) sky@MS97377:~/spark_exercises/ecommerce_project$ echo

(venv) sky@MS97377:~/spark_exercises/ecommerce_project$ echo "===== CUSTOMER LIFETIME VALUE ====="
===== CUSTOMER LIFETIME VALUE =====
(venv) sky@MS97377:~/spark_exercises/ecommerce_project$ cat output/customer_lifetime_value/part-*.csv
customer_id,name,Lifetime_Value,Orders
4,David,1800,2
3,Charlie,1400,2
2,Bob,1320,2
1,Alice,1160,2
5,Eva,660,2
(venv) sky@MS97377:~/spark_exercises/ecommerce_project$ find output -name "*.csv"
output/category_sales/part-00000-20de7302-2012-4636-9895-23083334750d-c000.csv
output/daily_revenue/part-00000-c5d65d39-d22b-4b4c-85f8-d82451661169-c000.csv
output/city_sales/part-00000-36ba8ee2-40fc-4728-af9a-54854b4cc88e-c000.csv
output/top_products/part-00000-366466c8-76e6-4ea6-aafc-f99d2a1525b3-c000.csv
output/customer_lifetime_value/part-00000-d559cc2f-80a0-4a9b-894b-db7d3c72a41f-c000.csv
output/top_customers/part-00000-f7b51611-3129-42b8-90e3-5d67c3507ed0-c000.csv
(venv) sky@MS97377:~/spark_exercises/ecommerce_project$ tree output
output
├── category_sales
│   ├── _SUCCESS
│   └── part-00000-20de7302-2012-4636-9895-23083334750d-c000.csv
├── city_sales
│   ├── _SUCCESS
│   └── part-00000-36ba8ee2-40fc-4728-af9a-54854b4cc88e-c000.csv
├── customer_lifetime_value
│   ├── _SUCCESS
│   └── part-00000-d559cc2f-80a0-4a9b-894b-db7d3c72a41f-c000.csv
├── daily_revenue
│   ├── _SUCCESS
│   └── part-00000-c5d65d39-d22b-4b4c-85f8-d82451661169-c000.csv
├── top_customers
│   ├── _SUCCESS
│   └── part-00000-f7b51611-3129-42b8-90e3-5d67c3507ed0-c000.csv
├── top_products
│   ├── _SUCCESS
│   └── part-00000-366466c8-76e6-4ea6-aafc-f99d2a1525b3-c000.csv
6 directories, 12 files
(venv) sky@MS97377:~/spark_exercises/ecommerce_project$

```

Conclusion

The E-Commerce Analytics Platform was successfully developed using Apache Spark and PySpark. The project demonstrates how distributed data processing can efficiently handle structured e-commerce data while performing ETL operations, data cleaning, and business analytics. Various analytical reports such as daily revenue, sales by category, city-wise sales, top-selling products, top customers, and customer lifetime value were generated successfully. Performance optimization techniques including Spark Cache and Broadcast Join improved processing efficiency, and the analytical results were exported as CSV reports. Overall, this project showcases the practical application of Apache Spark for scalable data processing and business intelligence in an e-commerce environment.